

UT.6.2 PROGRAMACIÓN EN BASES DE DATOS: DATOS COMPUESTOS Y CURSORES

1. INTRODUCCIÓN	2
2. TIPOS DE DATOS COMPUESTOS	2
2.1 REGISTROS	2
2.1.1 EL ATRIBUTO %ROWTYPE Y SU RELACIÓN CON LOS REGISTROS	5
2.2 COLECCIONES. ARRAYS DE LONGITUD VARIABLE	6
2.2.1 ARRAYS DE LONGITUD VARIABLE	6
2.3 COLECCIONES. TABLAS ANIDADAS	13
2.4 COLECCIONES. ARRAYS ASOCIATIVOS (TABLAS INDEXADAS)	19
3. CURSORES	21
3.1 CURSORES IMPLÍCITOS	22
3.2 CURSORES EXPLÍCITOS	22
3.3 ATRIBUTOS DE CURSORES	26
3.4 BUCLES FOR DE CURSOR	27
3.4 CURSORES CON PARÁMETROS	29

1.INTRODUCCIÓN

Una vez dominados los fundamentos de PL/SQL, tales como la declaración de variables simples y el uso de estructuras de control (condicionales y bucles), es necesario dar un paso hacia la construcción de aplicaciones más robustas y eficientes. En esta segunda parte del módulo, exploraremos herramientas avanzadas que permiten modelar datos complejos y optimizar la interacción entre el código procedimental y la base de datos relacional.

El contenido de este capítulo se centra en tres pilares fundamentales que transforman la forma en que manipulamos la información:

1. **Los registros (RECORD):** Aprenderemos a agrupar elementos relacionados bajo un único nombre, facilitando la organización de la información y permitiendo el tratamiento de filas completas de la base de datos de manera atómica mediante el atributo %ROWTYPE.
2. **Las colecciones:** Analizaremos las distintas estructuras (VARARRAYs, tablas anidadas y arrays asociativos) que permiten gestionar conjuntos de datos directamente en la memoria del programa. Estas estructuras son vitales para mejorar el rendimiento de los scripts mediante el procesamiento masivo de datos.
3. **Los cursores:** Estudiaremos el mecanismo esencial que actúa como puente entre el motor SQL y PL/SQL. Los cursores nos permitirán recuperar, navegar y procesar múltiples registros de las tablas de forma controlada y eficiente.

El objetivo de esta unidad es dar las herramientas para manejar volúmenes de información complejos con eficiencia, sentando las bases definitivas para la creación de subprogramas (procedimientos y funciones) y disparadores (*triggers*) en entornos profesionales.

2.TIPOS DE DATOS COMPUESTOS

En capítulos anteriores, entre otras cosas, hemos conocido los tipos de datos simples con los que cuenta PL/SQL. Pero dependiendo de la complejidad de los problemas, necesitamos disponer de otras estructuras en las que apoyarnos para poder modelar nuestro problema. En este capítulo nos vamos a centrar en conocer los tipos de datos complejos que nos ofrece PL/SQL y cómo utilizarlos para así poder sacarle el mayor partido posible.

2.1 REGISTROS

El uso de los registros es algo muy común en los lenguajes de programación. PL/SQL también nos ofrece este tipo de datos. En este apartado veremos qué son y cómo definirlos y utilizarlos.

Un registro es un grupo de elementos relacionados, referenciados por un único nombre, almacenados en campos, cada uno de los cuales tiene su propio nombre y tipo de dato. Por ejemplo, una dirección podría ser un registro con campos como calle, número, piso,

puerta, código postal, ciudad, provincia y país. Los registros hacen que la información sea más fácil de organizar y representar. Para declarar un registro seguiremos la siguiente sintaxis:

```
TYPE nombre_tipo IS RECORD (decl_campo[, decl_campo] ...);
```

donde:

```
decl_campo := nombre_tipo [[NOT NULL] {:=|DEFAULT} expresion]
```

El tipo del campo será cualquier tipo de dato válido en PL/SQL excepto REF CURSOR. La expresión será cualquier expresión que evalúe al tipo de dato del campo. Ejemplo:

```
DECLARE
    TYPE direccion IS RECORD
    (
        calle VARCHAR2(50),
        numero NUMBER(4),
        piso NUMBER(4),
        puerta VARCHAR2(2),
        codigo_postal NUMBER(5),
        ciudad VARCHAR2(30),
        provincia VARCHAR2(20),
        pais VARCHAR2(20) := 'España'
    );
    -- Declaro una variable de ese tipo
    mi_direccion direccion;
BEGIN
    NULL;
END;
/
```

Para acceder a los campos usaremos el operador punto:

```
DECLARE
TYPE direccion IS RECORD
(
    calle VARCHAR2(50),
    numero NUMBER(4),
    piso NUMBER(4),
    puerta VARCHAR2(2),
    codigo_postal INTEGER(5),
    ciudad VARCHAR2(30),
    provincia VARCHAR2(20),
```

```

    pais VARCHAR2(20) := 'España'
);
-- Declaro una variable de ese tipo
mi_direccion direccion;
BEGIN
    mi_direccion.calle := 'Gran Vía';
    mi_direccion.numero := 15;
END;
/

```

Para asignar un registro a otro, éstos deben ser del mismo tipo, no basta que tengan el mismo número de campos y éstos emparejen uno a uno. Tampoco podemos comparar registros aunque sean del mismo tipo (habría que hacerlo campo a campo mediante operadores lógicos), ni tampoco comprobar si éstos son nulos. Podemos hacer SELECT en registros (podemos cargar el resultado de una SELECT en un registro). Ejemplo:

```

DECLARE
TYPE t_dep IS RECORD (
    id    DEPARTAMENTO.CODDEP%TYPE,
    nom    DEPARTAMENTO.NOMDEP%TYPE
);

v_d t_dep;

BEGIN
    SELECT CODDEP, NOMDEP
    INTO v_d
    FROM DEPARTAMENTO
    WHERE CODDEP = 'MARKG';

    DBMS_OUTPUT.PUT_LINE(v_d.id || ' ' || v_d.nom);
END;
/

```

Desde Oracle 9i r2, se pueden realizar inserciones y actualizaciones de filas completas de una manera muy limpia desde un registro. Veamos un ejemplo práctico:

```

DECLARE
-- Declaramos un registro basado en la estructura de la tabla
v_dep DEPARTAMENTO%ROWTYPE;
BEGIN
-- 1. PREPARAR LOS DATOS PARA INSERTAR
v_dep.CODDEP := 'MKGDI';

```

```

v_dep.NOMDEP := 'Marketing digital';
v_dep.PREANU := 150000;

-- Inserción de la fila completa
INSERT INTO DEPARTAMENTO VALUES v_dep;
DBMS_OUTPUT.PUT_LINE('Departamento insertado con éxito.');
```

-- 2. MODIFICAR DATOS EXISTENTES

-- Supongamos que queremos subir el presupuesto y cambiar el nombre

```

v_dep.NOMDEP := 'Marketing digital y RRSS';
v_dep.PREANU := 250000;
```

-- Actualización de la fila completa usando SET ROW

```

UPDATE DEPARTAMENTO
SET ROW = v_dep
WHERE CODDEP = 'MKGDI';

DBMS_OUTPUT.PUT_LINE('Fila actualizada usando el registro
completo.');
```

```

COMMIT;
END;
/
```

2.1.1 EL ATRIBUTO %ROWTYPE Y SU RELACIÓN CON LOS REGISTROS

El atributo **%ROWTYPE** es una extensión fundamental de PL/SQL que simplifica la manipulación de filas de datos completas de una manera robusta y de fácil mantenimiento. Su propósito es declarar una variable de tipo **registro** ('**RECORD**') que hereda automáticamente la estructura de una fila de una tabla o de una fila resultante de un cursor.

Cuando se utiliza **%ROWTYPE**, el compilador de PL/SQL crea internamente una variable de tipo registro, donde cada campo dentro de ese registro corresponde:

1. **A una columna de una tabla o vista de la base de datos** (la forma más común).
2. **A una columna de un cursor o subconsulta** (permitiendo trabajar con resultados de consultas complejas).

Relación con el tipo registro ('RECORD'): El uso de **%ROWTYPE** es, en esencia, la forma más habitual y recomendada de **declarar un tipo de registro implícito** en PL/SQL. En lugar de definir manualmente cada campo del registro (como se haría con la declaración explícita de un tipo **RECORD**), **%ROWTYPE** lo define por ti, garantizando una total correspondencia de nombres y tipos de datos con la estructura de la base de datos.

Ventajas:

- **Mantenibilidad:** Si la estructura de la tabla subyacente cambia (se añade, elimina o modifica una columna), la variable declarada con **%ROWTYPE** se adapta automáticamente al regenerarse el bloque, sin necesidad de modificar el código.
- **Seguridad de tipos:** Garantiza que la variable local tenga el tipo de dato y el tamaño exacto de la columna correspondiente, previniendo errores de truncamiento o conversión en tiempo de ejecución.

Ejemplo de uso

```
DECLARE
    -- v_departamento es una variable de tipo registro con la estructura
    de la tabla departamento
    v_departamento departamento%ROWTYPE;
BEGIN
    -- Recuperar todos los campos de un departamento en una sola
    variable.
    SELECT *
    INTO v_departamento
    FROM departamento
    WHERE CodDep = 'JEFZS';

    -- Acceder a los campos del registro usando la notación punto.
    DBMS_OUTPUT.PUT_LINE('Nombre del departamento: ' ||
v_departamento.NomDep);
    DBMS_OUTPUT.PUT_LINE('Presupuesto anual: ' || v_departamento.PreAnu);
END;
/
```

2.2 COLECCIONES. ARRAYS DE LONGITUD VARIABLE

Una **colección** es un grupo ordenado de elementos, todos del mismo tipo. Cada elemento tiene un índice único que determina su posición en la colección.

En PL/SQL las colecciones sólo pueden tener una dimensión, mientras que disponemos de 2 clases de colecciones: arrays de longitud variable y tablas anidadas.

2.2.1 ARRAYS DE LONGITUD VARIABLE

Los elementos del tipo **VARRAY** son los llamados **arrays de longitud variable**. Son como los arrays de cualquier otro lenguaje de programación, pero con la salvedad de que a la hora de declararlos indicamos su tamaño máximo. y el array podrá ir creciendo dinámicamente hasta alcanzar ese tamaño. Un VARRAY siempre tiene un límite inferior igual a 1 y un límite superior igual al tamaño máximo. Para poder crear variables de tipo VARRAY primero tenemos que declarar un tipo nuevo para el tipo de datos que almacenará el array:

```
TYPE nombre IS {VARRAY | VARYING} (tamaño_máximo) OF tipo_elementos
[NOT NULL];
```

Donde **tamaño_máximo** es un número entero positivo que limita la cantidad de elementos. **tipo_elementos** debe ser un tipo de dato **simple** (como NUMBER, VARCHAR2 o DATE). Si decides usar un **registro** (RECORD), éste no puede contener otras colecciones dentro; debe estar compuesto únicamente por campos simples.

Una vez creado el tipo podremos empezar a crear variables de ese nuevo tipo.

```
DECLARE
    -- 1. Definimos el TIPO (el molde)
    TYPE t_telefonos IS VARRAY(3) OF VARCHAR2(15);

    -- 2. Declaramos la VARIABLE (el objeto real)
    v_mis_numeros t_telefonos;
BEGIN
    -- Aquí ya podemos usar v_mis_numeros
    NULL;
END;
```

Cuando creamos un VARRAY, éste es automáticamente nulo, por lo que para empezar a utilizarlo deberemos inicializarlo. Para ello debemos llamar al **constructor**, que es una función del sistema que tiene el **mismo nombre que el tipo (TYPE)**. Su función es sacar a la colección de su estado "atómica nula" (inexistente) y reservarle un espacio en la memoria. Ejemplo:

```
DECLARE

    TYPE ARRAY_DEPARTAMENTOS IS VARRAY(100) OF departamento%ROWTYPE; --
    100 será el tamaño MÁXIMO de los arrays de este tipo
    lista_departamentos ARRAY_DEPARTAMENTOS := ARRAY_DEPARTAMENTOS(); --
    Creo el array con un constructor sin parámetros. Ahora mismo el tamaño
    del array es 0

    TYPE ARRAY_NUMEROS IS VARRAY(10) OF NUMBER; -- 10 será el tamaño
    MÁXIMO de los arrays de este tipo
    lista_numeros ARRAY_NUMEROS := ARRAY_NUMEROS(5); -- En este caso el
    tamaño del array es 1, y el valor de el valor con índice 1 del array es
    5

BEGIN
    FOR i IN 2..5 LOOP
```

```

        lista_numeros.EXTEND; -- Amplió en 1 el tamaño del array. Esta
operación es OBLIGATORIA previamente a poder almacenar un nuevo valor
        lista_numeros(i) := i; -- Valorizo el nuevo elemento del array
que acabamos de crear
    END LOOP;

    lista_numeros.EXTEND(5); -- Así creamos 5 nuevos elementos en el
array, que estarán vacíos por ahora.
    --lista_numeros.EXTEND(10); -- OJO!! Esto fallará porque el tamaño
máximo del array es 10 y ya tenemos las 10 posiciones creadas
    FOR i IN 1..5 LOOP
        DBMS_OUTPUT.PUT_LINE('Elemento con índice' || i || ': '
|| lista_numeros(i));
    END LOOP;

    lista_departamentos.EXTEND; -- Este array tenía tamaño 0. Ahora
podemos almacenar un valor.
    SELECT * INTO lista_departamentos(1)
    FROM DEPARTAMENTO
    WHERE CODDEP = 'VENZS';
    DBMS_OUTPUT.PUT_LINE(lista_departamentos(1).NomDep);
END;
/

```

Como ves en el ejemplo anterior, para referenciar elementos en un VARRAY utilizaremos la sintaxis nombre_colección(subíndice). De igual modo puedes ver que para extender un VARRAY usaremos el método EXTEND. Sin parámetros, extendemos en 1 elemento nulo el VARRAY. EXTEND(n) añade n elementos nulos al VARRAY y EXTEND(n, i) añade n copias del i-ésimo elemento.

COUNT nos dirá el número de elementos del VARRAY. LIMIT nos dice el tamaño máximo del VARRAY. FIRST siempre será 1. LAST siempre será igual a COUNT. PRIOR(n) y NEXT(n) devolverá el antecesor y el sucesor del elemento. TRIM(n) elimina n elementos del final del array, y DELETE vacía el array por completo. Éstas últimas instrucciones **eliminan físicamente la casilla** de la memoria activa por lo que será necesario volver a hacer EXTEND para añadir elementos.

Al trabajar con VARRAY podemos hacer que salte alguna de las siguientes excepciones, debidas a un mal uso de los mismos: COLLECTION_IS_NULL, SUBSCRIPT_BEYOND_COUNT, SUBSCRIPT_OUTSIDE_LIMIT y VALUE_ERROR. Veremos la gestión de excepciones en PL/SQL más adelante.

Ejemplo de creación y uso de EXTEND:


```

DECLARE
    TYPE tab_num IS VARRAY(10) OF NUMBER;
    mi_tab tab_num;
BEGIN
    mi_tab := tab_num();
    FOR i IN 1..10 LOOP
        mi_tab.EXTEND;
        mi_tab(i) := POWER(i, 2);
    END LOOP;
    FOR i IN 1..10 LOOP
        DBMS_OUTPUT.PUT_LINE(i || ' al cuadrado es ' || mi_tab(i));
    END LOOP;
END;

```

Ejemplo de uso de propiedades de VARRAY:

```

DECLARE
    TYPE numeros IS VARRAY(20) OF NUMBER;
    tabla_numeros numeros := numeros();
BEGIN
    DBMS_OUTPUT.PUT_LINE(tabla_numeros.COUNT); -- 0
    FOR i IN 1..10 LOOP
        tabla_numeros.EXTEND;
        tabla_numeros(i) := i;
    END LOOP;
    DBMS_OUTPUT.PUT_LINE(tabla_numeros.COUNT); -- 10
    DBMS_OUTPUT.PUT_LINE(tabla_numeros.LIMIT); -- 20
    DBMS_OUTPUT.PUT_LINE(tabla_numeros.FIRST); -- 1
    DBMS_OUTPUT.PUT_LINE(tabla_numeros.LAST); -- 10
END;

```

Ejemplo de uso de PRIOR y NEXT, TRIM y DELETE:

```

DECLARE
    TYPE t_notas IS VARRAY(5) OF NUMBER;
    v_notas t_notas := t_notas(10, 8, 7, 9); -- Empezamos con 4
    elementos
BEGIN
    -- 1. Uso de PRIOR y NEXT
    DBMS_OUTPUT.PUT_LINE('El anterior al 3 es: ' || v_notas.PRIOR(3));
-- 2
    DBMS_OUTPUT.PUT_LINE('El siguiente al 3 es: ' || v_notas.NEXT(3));
-- 4

```

```

-- 2. Intentar borrar el elemento 2 (;ERROR!)
-- v_notas.DELETE(2); -- Esto lanzaría un error en VARRAY

-- 3. Recortar el array (Quitar el 9 y el 7)
v_notas.TRIM(2);

DBMS_OUTPUT.PUT_LINE('Ahora el último es: ' ||
v_notas(v_notas.LAST)); -- Imprimirá 8
DBMS_OUTPUT.PUT_LINE('Nuevo conteo: ' || v_notas.COUNT);
-- Imprimirá 2

-- 4. Borrar todo
v_notas.DELETE;
DBMS_OUTPUT.PUT_LINE('¿Está vacío?: ' || v_notas.COUNT); --
Imprimirá 0
END;

```

Posibles excepciones:

```

DECLARE
    TYPE numeros IS VARRAY(20) OF INTEGER;
    v_numeros numeros := numeros( 10, 20, 30, 40 );
    v_enteros numeros;
BEGIN
    v_enteros(1) := 15; --lanzaría COLECTION_IS_NULL
    v_numeros(5) := 20; --lanzaría SUBSCRIPT_BEYOND_COUNT
    v_numeros(-1) := 5; --lanzaría SUBSCRIPT_OUTSIDE_LIMIT
    v_numeros('A') := 25; --lanzaría VALUE_ERROR
END;

```

Ejercicios:

1. Debes simular el cálculo de una factura simplificada. Para ello:
 - Define un **registro** llamado t_producto con los campos: nombre (texto), precio (número) y cantidad (entero).
 - Define un **VARRAY** de capacidad 5 que almacene registros de tipo t_producto.
 - Crea un bloque que:
 - Inicialice el array con 3 productos de tu elección.
 - Recorra el array para calcular el **subtotal** de la compra.
 - Aplique un **descuento del 10%** si el subtotal supera los **100€**.
 - Muestre por pantalla el nombre de cada producto y, al final, el total a pagar.

2. Sistema de asignación de aulas. Crear un sistema que busca el aula más pequeña disponible que cumpla con el aforo necesario. Para ello:
- Define un **registro** t_aula con: id_aula (número), nombre (texto) y capacidad (número).
 - Define un **VARRAY** de capacidad 10 de este registro.
 - Crea un bloque que:
 - Rellene el array con 4 aulas de capacidades: 20, 50, 35 y 100.
 - Dada una variable v_alumnos := 40, el script debe recorrer el array y encontrar **la primera aula** donde quepan todos los alumnos.
 - Si la encuentra, debe imprimir: *"Asignada el aula [Nombre] (Capacidad: [X])"*.
 - Si recorre todo el array y ninguna es apta, debe imprimir: *"No hay aulas disponibles para ese aforo"*.

Soluciones:

```
DECLARE
    TYPE t_producto IS RECORD (
        nombre    VARCHAR2(50),
        precio    NUMBER,
        cantidad  INTEGER
    );
    TYPE t_carrito IS VARRAY(5) OF t_producto;

    mi_compra t_carrito := t_carrito();
    v_subtotal NUMBER := 0;
    v_total    NUMBER := 0;
BEGIN
    -- Inicialización y carga de datos
    mi_compra.EXTEND(3);
    mi_compra(1).nombre := 'Monitor 24"; mi_compra(1).precio := 120;
mi_compra(1).cantidad := 1;
    mi_compra(2).nombre := 'Ratón Óptico'; mi_compra(2).precio := 15;
mi_compra(2).cantidad := 2;
    mi_compra(3).nombre := 'Alfombrilla';  mi_compra(3).precio := 10;
mi_compra(3).cantidad := 1;

    -- Procesamiento
    FOR i IN 1..mi_compra.COUNT LOOP
        DBMS_OUTPUT.PUT_LINE('Producto: ' || mi_compra(i).nombre || ' |
Cant: ' || mi_compra(i).cantidad);
        v_subtotal := v_subtotal + (mi_compra(i).precio *
mi_compra(i).cantidad);
    END LOOP;
```

```

v_total := v_subtotal;

-- Lógica de descuento
IF v_subtotal > 100 THEN
    v_total := v_subtotal * 0.90;
    DBMS_OUTPUT.PUT_LINE('Se ha aplicado un descuento del 10%.');
END IF;

DBMS_OUTPUT.PUT_LINE('Total a pagar: ' || v_total || '€');
END;
/

```

```

DECLARE
    TYPE t_aula IS RECORD (
        id_aula    NUMBER,
        nombre     VARCHAR2(20),
        capacidad  NUMBER
    );
    TYPE t_lista_aulas IS VARRAY(10) OF t_aula;

    v_aulas    t_lista_aulas := t_lista_aulas();
    v_alumnos  NUMBER := 40;
    v_hallada  BOOLEAN := FALSE;
BEGIN
    -- Carga de aulas
    v_aulas.EXTEND(4);
    v_aulas(1) := t_aula(101, 'Aula A', 20);
    v_aulas(2) := t_aula(102, 'Aula B', 50);
    v_aulas(3) := t_aula(103, 'Aula C', 35);
    v_aulas(4) := t_aula(104, 'Aula D', 100);

    -- Búsqueda
    FOR i IN 1..v_aulas.COUNT LOOP
        IF v_aulas(i).capacidad >= v_alumnos THEN
            DBMS_OUTPUT.PUT_LINE('Asignada el ' || v_aulas(i).nombre ||
                                ' (Capacidad: ' || v_aulas(i).capacidad
                                || ')');
            v_hallada := TRUE;
            EXIT; -- Salimos del bucle al encontrar la primera
        END IF;
    END LOOP;

    IF NOT v_hallada THEN

```

```

        DBMS_OUTPUT.PUT_LINE('No hay aulas disponibles para ese
aforo.');
```

```

    END IF;
END;
/
```

A diferencia de los VARRAYs, que son siempre contiguos (densos), las tablas anidadas que veremos a continuación permiten 'huecos' en su estructura, lo que cambia radicalmente la forma de recorrerlas.

2.3 COLECCIONES. TABLAS ANIDADAS

Las tablas anidadas son colecciones de elementos, que no tienen límite superior fijo, y pueden aumentar dinámicamente su tamaño. Además, permiten eliminar elementos individuales. Para declararlas utilizaremos la siguiente sintaxis:

```
TYPE nombre IS TABLE OF tipo_elementos [NOT NULL];
```

Donde tipo_elementos tendrá las mismas restricciones que para los VARRAY. Al igual que pasaba con los VARRAY, al declarar una tabla anidada, ésta es automáticamente nula al crearla, por lo que deberemos inicializarla antes de poder utilizarla:

```

TYPE t_numeros IS TABLE OF NUMBER;
v_nums t_numeros := t_numeros(10, 20, 30); -- Inicialización con
valores
v_vacia t_numeros := t_numeros(); -- Constructor nulo (vacía)
```

También podemos usar un constructor nulo. Para referenciar elementos usamos la misma sintaxis que para los VARRAY.

Para extender una tabla usamos EXTEND exactamente igual que para los VARRAY. COUNT nos dirá el número de elementos (sin tener en cuenta los "huecos"), que no tiene por qué coincidir con LAST.

```

DECLARE
    TYPE t_lista IS TABLE OF VARCHAR2(20);
    v_tabla t_lista := t_lista('A', 'B', 'C'); -- Índices 1, 2, 3
BEGIN
    v_tabla.DELETE(2); -- Borramos el elemento 'B' (el hueco)

    DBMS_OUTPUT.PUT_LINE('COUNT: ' || v_tabla.COUNT); -- Imprime 2 (Solo
quedan 'A' y 'C')
    DBMS_OUTPUT.PUT_LINE('LAST: ' || v_tabla.LAST); -- Imprime 3 (El
último sigue siendo el 3)
```

```

-- ¿Qué pasa si borramos el último?
v_tabla.DELETE(3);
DBMS_OUTPUT.PUT_LINE('Ahora LAST es: ' || v_tabla.LAST); -- Imprime
1
END;
/

```

LIMIT no tiene sentido y devuelve NULL. EXISTS(n) devuelve TRUE si el elemento existe, y FALSE en otro caso (el elemento ha sido borrado). FIRST devuelve el índice del primer elemento, que no siempre será 1 porque hemos podido borrar elementos del principio. LAST devuelve el índice del último elemento. PRIOR y NEXT nos dicen el antecesor y sucesor del elemento (ignorando elementos borrados). TRIM sin argumentos borra un elemento del final de la tabla. TRIM(n) borra n elementos del final de la tabla. TRIM opera en el tamaño interno, por lo que si encuentra un elemento borrado con DELETE, lo incluye para ser eliminado de la colección. DELETE(n) borra el n-ésimo elemento. DELETE(n, m) borra del elemento n al m. Si después de hacer DELETE, consultamos si el elemento existe nos devolverá FALSE.

Al trabajar con tablas anidadas podemos hacer que salte alguna de las siguientes excepciones, debidas a un mal uso de las mismas: COLLECTION_IS_NULL, NO_DATA_FOUND (Salta si intentas acceder a un índice que existía pero que ha sido borrado con DELETE), SUBSCRIPT_BEYOND_COUNT y VALUE_ERROR.

Ejemplo:

```

SET SERVEROUTPUT ON;

DECLARE
    TYPE numeros IS TABLE OF NUMBER;
    tabla_numeros numeros := numeros();
    num NUMBER;
BEGIN
    -- 1. Estado inicial
    num := tabla_numeros.COUNT;
    DBMS_OUTPUT.PUT_LINE('1. Inicialización - COUNT: ' || num); --
    Esperado: 0

    -- 2. Llenamos la tabla del 1 al 10
    FOR i IN 1..10 LOOP
        tabla_numeros.EXTEND;
    END LOOP;
END;
/

```

```

        tabla_numeros(i) := i;
    END LOOP;
    num := tabla_numeros.COUNT;
    DBMS_OUTPUT.PUT_LINE('2. Tras llenar 1..10 - COUNT: ' || num); --
Esperado: 10

    -- 3. Borramos el último (10)
    tabla_numeros.DELETE(10);
    num := tabla_numeros.LAST;
    DBMS_OUTPUT.PUT_LINE('3. Tras borrar el 10 - LAST: ' || num); --
Esperado: 9

    -- 4. Comprobamos el primero
    num := tabla_numeros.FIRST;
    DBMS_OUTPUT.PUT_LINE('4. El primero actual es el índice: ' || num);
-- Esperado: 1

    -- 5. Borramos el primero (1) y vemos cómo cambia FIRST
    tabla_numeros.DELETE(1);
    num := tabla_numeros.FIRST;
    DBMS_OUTPUT.PUT_LINE('5. Tras borrar el 1 - El nuevo FIRST es: ' ||
num); -- Esperado: 2

    -- 6. Borramos los pares (2, 4, 6, 8)
    FOR i IN 1..4 LOOP
        tabla_numeros.DELETE(2*i);
    END LOOP;

    -- 7. Recuento final y posición del último
    num := tabla_numeros.COUNT;
    DBMS_OUTPUT.PUT_LINE('6. Tras borrar pares - COUNT (elementos
vivos): ' || num); -- Esperado: 4 (quedan 3, 5, 7, 9)

    num := tabla_numeros.LAST;
    DBMS_OUTPUT.PUT_LINE('7. Tras borrar pares - LAST (índice final): '
|| num); -- Esperado: 9
END;
/

```

Excepciones:

```

DECLARE
    TYPE numeros IS TABLE OF NUMBER;
    tabla_num numeros := numeros();
    tabla1 numeros;
BEGIN
    tabla1(5) := 0;
    --lanzaría COLLECTION_IS_NULL
    tabla_num.EXTEND(5);
    tabla_num.DELETE(4);
    tabla_num(4) := 3;
    --lanzaría NO_DATA_FOUND
    tabla_num(6) := 10;
    --lanzaría SUBSCRIPT_BEYOND_COUNT
    tabla_num(-1) := 0;
    --lanzaría SUBSCRIPT_OUTSIDE_LIMIT
    tabla_num('y') := 5; --lanzaría VALUE_ERROR
END;

```

Ejercicios:

- Realiza un algoritmo de “compactación” para una lista anidada. Para ello:
 - Definición:** Crea un tipo de dato de tabla anidada que almacene cadenas de texto (VARCHAR2).
 - Carga Inicial:** Declara una variable de ese tipo e inicialízala con 5 letras.
 - Simulación de bajas:** Elimina de forma manual las posiciones **2 y 4**.
 - Diagnóstico:** Muestra por pantalla el valor de `.COUNT` y el valor de `.LAST` para evidenciar que ya no coinciden.
 - Algoritmo de compactación:** Crea una segunda colección vacía (del mismo tipo).
 - Utiliza un bucle que recorra la colección original saltando los huecos.
 - Copia los elementos existentes a la nueva colección de forma que queden contiguos (sin saltos de índice).
 - Resultado final:** Sustituye la colección original por la nueva y muestra de nuevo `.COUNT` y `.LAST` para verificar que ahora son iguales.
- Una empresa tecnológica mantiene una lista de candidatos en una **tabla anidada**. Necesitan realizar una limpieza de los candidatos que no llegan a la nota de corte y, posteriormente, guardar al mejor candidato en una tabla de "candidatos top" de la base de datos.
 - Estructura de datos:**
 - Define un registro `t_candidato` con: nombre (texto), puntuacion (número entre 0 y 100) y tecnologia (texto).
 - Define una **tabla anidada** de este tipo de registro.

- **Carga y filtrado:**
 - Inicializa la colección con 5 candidatos.
 - **Proceso de cribado:** Recorre la colección y **elimina (DELETE)** a todo aquel candidato cuya puntuación sea inferior a **70**. Esto generará huecos en la tabla anidada.
- **Búsqueda del mejor:**
 - Una vez filtrada la lista, recorre los candidatos restantes para encontrar quién tiene la **puntuación más alta**.
 - Muestra por consola el nombre del ganador y cuántos candidatos han sobrevivido al filtro (usando `.COUNT`).
- **Persistencia (Simulacro %ROWTYPE):**
 - Supongamos que existe una tabla llamada `CANDIDATOS_TOP`.
 - Declara una variable `v_ganador` usando **`CANDIDATOS_TOP%ROWTYPE`**.
 - Vuelca los datos del mejor candidato de tu colección a esa variable e inserta la fila completa en la tabla (puedes dejar el código del `INSERT` comentado).

Soluciones:

```
DECLARE
    TYPE t_lista IS TABLE OF VARCHAR2(20);
    v_original t_lista := t_lista('A', 'B', 'C', 'D', 'E');
    v_compacta t_lista := t_lista(); -- Nueva colección vacía
    v_idx      INTEGER;
BEGIN
    -- 1. Creamos huecos artificiales
    v_original.DELETE(2);
    v_original.DELETE(4);
    -- Ahora v_original tiene: (1=>'A', 3=>'C', 5=>'E') -> COUNT: 3,
    LAST: 5
    DBMS_OUTPUT.PUT_LINE('Original:');
    DBMS_OUTPUT.PUT_LINE('COUNT: ' || v_original.COUNT); -- 3
    DBMS_OUTPUT.PUT_LINE('LAST: ' || v_original.LAST); -- 5

    -- 2. Proceso de compactación
    v_idx := v_original.FIRST;
    WHILE v_idx IS NOT NULL LOOP
        v_compacta.EXTEND;
        v_compacta(v_compacta.LAST) := v_original(v_idx);
        v_idx := v_original.NEXT(v_idx);
    END LOOP;
```

```

-- 3. Intercambio (Ahora v_original vuelve a ser densa)
v_original := v_compacta;

DBMS_OUTPUT.PUT_LINE('Tras compactar:');
DBMS_OUTPUT.PUT_LINE('COUNT: ' || v_original.COUNT); -- 3
DBMS_OUTPUT.PUT_LINE('LAST:   ' || v_original.LAST);  -- 3
(¡Iguales!)
END;
/

```

```

DECLARE
    -- 1. Definición de tipos
    TYPE t_candidato IS RECORD (
        nombre          VARCHAR2(50),
        puntuacion       NUMBER,
        tecnologia       VARCHAR2(20)
    );
    TYPE t_lista_candidatos IS TABLE OF t_candidato;

    v_lista t_lista_candidatos := t_lista_candidatos();
    v_top   t_candidato;
    v_idx   INTEGER;

    -- Para el punto 4 (Simulando que la tabla existe)
    -- v_final CANDIDATOS_TOP%ROWTYPE;

BEGIN
    -- 2. Carga inicial
    v_lista.EXTEND(5);
    v_lista(1) := t_candidato('Ana', 85, 'Java');
    v_lista(2) := t_candidato('Pedro', 60, 'Python'); -- Será eliminado
    v_lista(3) := t_candidato('Marta', 92, 'PL/SQL');
    v_lista(4) := t_candidato('Luis', 45, 'C++');      -- Será eliminado
    v_lista(5) := t_candidato('Sonia', 78, 'Java');

    -- 3. Proceso de Cribado (Generando huecos)
    FOR i IN 1..v_lista.LAST LOOP
        IF v_lista(i).puntuacion < 70 THEN
            v_lista.DELETE(i);
        END IF;
    END LOOP;

```

```

-- 4. Búsqueda del mejor (Navegando entre huecos)
v_idx := v_lista.FIRST;
v_top.puntuacion := -1; -- Inicializamos con valor bajo

WHILE v_idx IS NOT NULL LOOP
    IF v_lista(v_idx).puntuacion > v_top.puntuacion THEN
        v_top := v_lista(v_idx);
    END IF;
    v_idx := v_lista.NEXT(v_idx);
END LOOP;

-- 5. Resultados
DBMS_OUTPUT.PUT_LINE('Sobrevivientes: ' || v_lista.COUNT);
DBMS_OUTPUT.PUT_LINE('El mejor candidato es: ' || v_top.nombre || '
con ' || v_top.puntuacion || ' puntos.');
```

```

/* -- 6. Punto extra: Volcado a %ROWTYPE e INSERT
v_final.nombre := v_top.nombre;
v_final.puntos := v_top.puntuacion;
...
INSERT INTO CANDIDATOS_TOP VALUES v_final;
*/
END;
/
```

2.4 COLECCIONES. ARRAYS ASOCIATIVOS (TABLAS INDEXADAS)

A diferencia de los VARRAY y las tablas anidadas, los arrays asociativos (conocidos tradicionalmente como tablas indexadas) son colecciones que solo existen de forma temporal en la memoria de nuestro programa (no se pueden almacenar en la base de datos).

Su gran ventaja es que funcionan como un **diccionario o mapa**: nos permiten indexar los elementos por números no correlativos o incluso por **cadenas de texto (VARCHAR2)**.

Sintaxis de declaración:

```
TYPE nombre IS TABLE OF tipo_elementos [NOT NULL] INDEX BY
{PLS_INTEGER | VARCHAR2(tamaño)};
```

Características principales:

- **Sin Constructor:** No necesitan ser inicializados con `tipo()`. Empiezan a existir en cuanto se declaran.
- **Sin EXTEND:** No necesitan reservar espacio. Al asignar un valor a un índice, PL/SQL crea la "casilla" automáticamente: `v_tabla('clave') := valor;`.
- **Eficiencia:** Son ideales para tablas de búsqueda rápida o para almacenar datos temporales durante la ejecución de un proceso complejo.

Ejemplo de uso (Índices de texto):

```
DECLARE
    -- Definimos un array asociativo donde la clave es el nombre de un país
    TYPE t_paises IS TABLE OF VARCHAR2(50) INDEX BY VARCHAR2(20);
    v_capitales t_paises;
    v_indice VARCHAR2(20);
BEGIN
    -- No hace falta inicializar ni hacer EXTEND
    v_capitales('España') := 'Madrid';
    v_capitales('Francia') := 'París';
    v_capitales('Portugal') := 'Lisboa';

    -- Acceso directo
    DBMS_OUTPUT.PUT_LINE('La capital de Francia es: ' ||
v_capitales('Francia'));

    -- Recorrido (se ordenan alfabéticamente por la clave)
    v_indice := v_capitales.FIRST;
    WHILE v_indice IS NOT NULL LOOP
        DBMS_OUTPUT.PUT_LINE('País: ' || v_indice || ' - Capital: ' ||
v_capitales(v_indice));
        v_indice := v_capitales.NEXT(v_indice);
    END LOOP;
END;
/
```

Comparativa de colecciones en PL/SQL

Característica	VARRAY	Tabla anidada (Nested table)	Tabla indexada (Associative array)
Tamaño máximo	Definido en la declaración (Límite fijo).	Dinámico (Sin límite teórico).	Dinámico (Sin límite teórico).

Persistencia	Se puede guardar en tablas de la DB.	Se puede guardar en tablas de la DB.	Solo existe en memoria PL/SQL (No en columnas).
Índice	Numérico (siempre empieza en 1).	Numérico (siempre empieza en 1).	Cualquier número o cadena (VARCHAR2).
Estado de memoria	Denso (siempre consecutivo).	Disperso (puede tener huecos).	Disperso (puede tener huecos).
Uso de EXTEND	Obligatorio para añadir.	Obligatorio para añadir.	No se usa (se crea al asignar).
Borrado	Solo TRIM o DELETE completo.	DELETE(i) permitido (deja hueco).	DELETE(i) permitido (deja hueco).

¿Cuándo elegir cada una?

1. Usar VARRAY cuando:

- El número máximo de elementos es conocido y pequeño (ej. los días de la semana, los meses, las 3 últimas notas).
- El orden de los elementos es importante y no va a cambiar.
- Se va a recuperar siempre el conjunto completo de datos de la base de datos de una sola vez.

2. Usar tabla anidada cuando:

- No se sabe cuántos elementos habrá (ej. los hijos de un empleado, las líneas de una factura).
- Se necesita borrar elementos de forma individual (crear huecos).
- Se quiere realizar consultas SQL directamente sobre la colección almacenada en la base de datos.

3. Usar tabla indexada (associative array) cuando:

- La colección es **temporal** y solo se usa durante la ejecución del bloque PL/SQL.
- Se necesita indexar por algo que no sea un número correlativo (ej. usar el DNI o un código de producto como índice: v_precios('PROD_A') := 15.50).
- Se requiere una tabla de búsqueda (lookup table) rápida en memoria para evitar ir continuamente a la base de datos.

3. CURSORES

En los apartados anteriores hemos visto algunos tipos de datos compuestos cuyo uso es común en otros lenguajes de programación. Sin embargo, en este apartado vamos a ver un tipo de dato, que aunque se puede asemejar a otros que ya conozcas, su uso es exclusivo en la programación de las bases de datos y que es el **cursor**.

Un **cursor** no es más que una **estructura que almacena el conjunto de filas devuelto por una consulta a la base de datos**. Básicamente, es un **puntero** a un área de memoria donde Oracle almacena el resultado de una consulta SQL. Imaginalo como un "vínculo" activo entre tu código y los datos.

Oracle usa áreas de trabajo para ejecutar sentencias SQL y almacenar la información procesada. Hay 2 clases de cursores: implícitos y explícitos. PL/SQL declara implícitamente un cursor para todas las sentencias SQL de manipulación de datos, incluyendo consultas que devuelven una sola fila. Para las consultas que devuelven más de una fila, se debe declarar explícitamente un cursor para procesar las filas individualmente.

En este primer apartado vamos a hablar de los cursores implícitos y de los atributos de un cursor (estos atributos tienen sentido con los cursores explícitos, pero los introducimos aquí para ir abriendo boca), para luego pasar a ver los cursores explícitos y terminaremos hablando de los cursores variables.

3.1 CURSORES IMPLÍCITOS

Oracle abre **implícitamente** un cursor para procesar **cada sentencia SQL** que no esté asociada con un cursor declarado explícitamente.

Con un cursor implícito no podemos usar las sentencias OPEN, FETCH y CLOSE para controlar el cursor. Pero sí podemos usar los atributos del cursor para obtener información sobre las sentencias SQL más recientemente ejecutadas. Veremos estos atributos más adelante, tras el apartado de cursores explícitos.

3.2 CURSORES EXPLÍCITOS

Cuando una consulta devuelve **múltiples filas**, debemos declarar **explícitamente un cursor** para procesar las filas devueltas. Cuando declaramos un cursor, lo que hacemos es darle un nombre y asociarle una consulta usando la siguiente sintaxis:

```
CURSOR nombre_cursor [(parametro [, parametro] ...)] [RETURN  
tipo_devuelto] IS sentencia_selección
```

Donde `tipo_devuelto` debe representar un registro o una fila de una tabla de la base de datos, y `parámetro` sigue la siguiente sintaxis:

```
parametro := nombre_parametro [IN] tipo_dato [{:= | DEFAULT}  
expresion]
```

Muchas ocasiones no necesitaremos parámetros, por lo que la declaración del cursor queda como:

```
CURSOR nb_cursor IS sentencia SELECT;
```

Ejemplos:

```
CURSOR cAgentes IS SELECT * FROM agentes;  
CURSOR cFamilias RETURN familias%ROWTYPE IS SELECT * FROM familias  
WHERE ...
```

Además, como hemos visto en la declaración, un cursor puede tomar parámetros, los cuales pueden aparecer en la consulta asociada como si fuesen constantes. Los parámetros serán de entrada, un cursor no puede devolver valores en los parámetros actuales. A un parámetro de un cursor no podemos imponerle la restricción NOT NULL.

```
CURSOR c1 (cat INTEGER DEFAULT 0) IS SELECT * FROM agentes WHERE  
categoria = cat;
```

La siguiente operación, tras la declaración, será la **apertura del cursor mediante la instrucción OPEN**. Cuando abrimos un cursor, lo que se hace es **ejecutar la consulta asociada e identificar el conjunto resultado**, que serán todas las filas que emparejen con el criterio de búsqueda de la consulta. Si la consulta no devuelve ninguna fila, no se producirá ninguna excepción. Para abrir un cursor usamos la sintaxis:

```
OPEN nombre_cursor [(parametro [, parametro] ...)];
```

En el caso de no tener parámetros será suficiente con hacer:

```
OPEN nb_cursor;
```

La siguiente operación será **recorrer el cursor** recuperando el resultado de la consulta fila a fila. La sentencia **FETCH** devuelve una fila del conjunto resultado. Después de cada **FETCH**, el cursor **avanza a la próxima fila** en el conjunto resultado.

```
FETCH nb_cursor INTO [ variable1, variable2, ... | nb_registro] ;
```

Para cada valor de columna devuelto por la consulta **SELECT** asociada al cursor, debe haber una variable que se corresponda en la lista de variables después del **INTO**.

Para procesar un cursor entero deberemos hacerlo por medio de un bucle. Una vez finalizado el trabajo con el cursor, **éste deberá ser cerrado**, para lo que se utiliza la orden **CLOSE**:

```
CLOSE nb_cursor;
```

Veamos unos ejercicios resueltos:

Ejercicio 1:

Declara un cursor que lea el nombre, salario y comisión de la tabla EMPLEADO de la base de datos EMPRESA.

Recorre el cursor mostrando el nombre del empleado, su salario y un literal diciendo si cobra o no comisión. Algo así:

Pérez Public, Ana - 3000 - Cobra comisión

Martínez Slogan, Luis - 5500 - No cobra comisión

Al final mostraremos la suma del salario de todos los empleados y la suma de comisiones de todos los empleados, junto con el número de empleados que cobran comisión. Todos estos valores se calcularán durante la ejecución del bucle con el que hemos recorrido el cursor.

El salario total de los empleados es de: 111400€

La comisión total de los empleados es de: 5020€ (16 empleados en total)

```
DECLARE
    CURSOR SALARIO IS
        SELECT NOMEMP, SALEMP, COMEMP
        FROM EMPLEADO;

    TEMPORAL SALARIO%ROWTYPE;
    SAL_TOTAL EMPLEADO.SALEMP%TYPE := 0;
    COM_TOTAL EMPLEADO.COMEMP%TYPE := 0;
    CONT_COM NUMBER(4) := 0;
BEGIN
    OPEN SALARIO;
    LOOP
        FETCH SALARIO INTO TEMPORAL;
        EXIT WHEN SALARIO%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(TEMPORAL.NOMEMP || ' - ' || TEMPORAL.SALEMP
||
        CASE
            WHEN TEMPORAL.COMEMP IS NOT NULL THEN ' - Cobra comisión'
            ELSE ' - No cobra comisión'
        END);
        SAL_TOTAL := SAL_TOTAL + TEMPORAL.SALEMP;
        IF (TEMPORAL.COMEMP IS NOT NULL) THEN
            COM_TOTAL := COM_TOTAL + TEMPORAL.COMEMP;
            CONT_COM := CONT_COM + 1;
        END IF;
    END LOOP;
```



```

CLOSE SALARIO;
    DBMS_OUTPUT.PUT_LINE('El salario total de los empleados es de: ' ||
SAL_TOTAL || '€');
    DBMS_OUTPUT.PUT_LINE('La comisión total de los empleados es de: ' ||
COM_TOTAL || '€ (' || CONT_COM || ' empleados en total)');
END;
/

```

Ejercicio 2:

Crear un bloque PL que acepte un número N como entrada, elija a los N empleados con el sueldo más alto y los inserte en la tabla TABLA_SALEMP, cuyo DDL es:

```

CREATE TABLE TABLA_SALEMP
(
    NOMEMP VARCHAR2(50),
    SALEMP NUMBER(7,2)
);

```

```

DECLARE
    CURSOR EMP_CURSOR IS
        SELECT NOMEMP, SALEMP
        FROM EMPLEADO
        ORDER BY SALEMP DESC;

    v_nomemp EMPLEADO.NOMEMP%TYPE;
    v_salemp EMPLEADO.SALEMP%TYPE;
    v_numero NUMBER(5) := &NUM_EMPLEADOS;
BEGIN
    OPEN EMP_CURSOR;
    FETCH EMP_CURSOR INTO v_nomemp, v_salemp;

    WHILE (EMP_CURSOR%ROWCOUNT <= v_numero)
    AND EMP_CURSOR%FOUND LOOP
        INSERT INTO TABLA_SALEMP (NOMEMP, SALEMP)
        VALUES (v_nomemp, v_salemp);
        FETCH EMP_CURSOR INTO v_nomemp, v_salemp;
    END LOOP;
    CLOSE EMP_CURSOR;
    COMMIT;
END;
/

```

3.3 ATRIBUTOS DE CURSORES

Cada cursor tiene 4 atributos que podemos usar para obtener información sobre la ejecución del mismo o sobre los datos. Estos atributos pueden ser usados en PL/SQL, pero no en SQL. Aunque estos atributos se refieren en general a cursores explícitos, aunque algunos también pueden utilizarse en implícitos.

%FOUND: Después de que el cursor esté abierto y antes del primer FETCH, %FOUND devuelve NULL. Después del primer FETCH, %FOUND devolverá TRUE si el último FETCH ha devuelto una fila y FALSE en caso contrario. Para cursores implícitos %FOUND devuelve TRUE si un INSERT, UPDATE o DELETE afectan a una o más de una fila, o un SELECT ... INTO ... devuelve una o más filas. En otro caso %FOUND devuelve FALSE.

```
LOOP
    FETCH CURSOR INTO VARIABLE
    IF CURSOR%FOUND THEN
        .....;
    ELSE
        EXIT;
    END IF;
END LOOP;
```

%NOTFOUND: Es lógicamente lo contrario a %FOUND.

```
LOOP
    FETCH CURSOR INTO VARIABLE;
    EXIT WHEN CURSOR%NOTFOUND;
END LOOP;
```

%ISOPEN: Evalúa a TRUE si el cursor está abierto y FALSE en caso contrario. Para cursores implícitos, como Oracle los cierra automáticamente, %ISOPEN evalúa siempre a FALSE.

```
IF CURSOR%ISOPEN THEN
    CLOSE CURSOR;
ELSE
    OPEN CURSOR;
END IF;
```

%ROWCOUNT: Para un cursor abierto y antes del primer FETCH, %ROWCOUNT evalúa a 0. Después de cada FETCH, %ROWCOUNT es incrementado y evalúa al número de filas que hemos procesado. Para cursores implícitos %ROWCOUNT evalúa al número de filas afectadas por un INSERT, UPDATE o DELETE o el número de filas devueltas por un SELECT ... INTO...

```

LOOP
    FETCH CURSOR INTO VARIABLE
    IF CURSOR%ROWCOUNT = 5 THEN
        EXIT;
    END IF;
END LOOP;

```

3.4 BUCLES FOR DE CURSOR

Un bucle FOR de cursor:

- Procesa filas en un cursor explícito.
- Abre, recupera y cierra de forma automática.
- No es necesario declarar la variable registro, ya que se declara implícitamente. La variable de registro **solo existe dentro del bucle**

Sintaxis:

```

FOR nb_registro IN nb_cursor LOOP
    órdenes;
    ...
END LOOP;

```

Ejemplo:

Recuperar, de uno en uno, todas las líneas de artículo pedido hasta que no queden más líneas, utilizando un bucle FOR de cursor.

```

.....
FOR PED_REG IN CURSOR_LIN LOOP
    V_TOTAL := V_TOTAL + (PED_REG.PRECIO*PED_REG.CANTIDAD);
    I:=I+1;
    TB_PRODUCTOS(I) := PED_REG.PRODUCTO;
    TB_TOTAL(I) := V_TOTAL;
END LOOP;
.....

```

Ejercicios resueltos:

Ejercicio 1:

Escribir un bloque PL/SQL que visualice el nombre y la fecha de alta de todos los empleados de la tabla EMPLEADO, ordenados ascendentemente por fecha de alta en la empresa. Hazlo utilizando un bucle LOOP, uno WHILE y un FOR.

Bucle LOOP

```

DECLARE
    CURSOR C_EMP IS
        SELECT NOMEMP, FECINEMP
        FROM EMPLEADO
        ORDER BY FECINEMP;
    V_NOMEMP EMPLEADO.NOMEMP%TYPE;
    V_FECINEMP EMPLEADO.FECINEMP%TYPE;
BEGIN
    OPEN C_EMP;
    LOOP
        FETCH C_EMP INTO V_NOMEMP, V_FECINEMP;
        EXIT WHEN C_EMP%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE (V_NOMEMP||': '||V_FECINEMP);
    END LOOP;
    CLOSE C_EMP;
END;
/

```

Bucle WHILE

```

DECLARE
    CURSOR C_EMP IS
        SELECT NOMEMP, FECINEMP
        FROM EMPLEADO
        ORDER BY FECINEMP;
    V_NOMEMP EMPLEADO.NOMEMP%TYPE;
    V_FECINEMP EMPLEADO.FECINEMP%TYPE;
BEGIN
    OPEN C_EMP;
    FETCH C_EMP INTO V_NOMEMP, V_FECINEMP;
    WHILE C_EMP%FOUND LOOP
        DBMS_OUTPUT.PUT_LINE (V_NOMEMP||': '||V_FECINEMP);
        FETCH C_EMP INTO V_NOMEMP, V_FECINEMP;
    END LOOP;
    CLOSE C_EMP;
END;
/

```

Bucle FOR

```

DECLARE
    CURSOR C_EMP IS
    SELECT NOMEMP, FECINEMP
    FROM EMPLEADO
    ORDER BY FECINEMP;
BEGIN
    FOR I IN C_EMP LOOP
        DBMS_OUTPUT.PUT_LINE (I.NOMEMP||': '||I.FECINEMP);
    END LOOP;
END;
/

```

3.4 CURSORES CON PARÁMETROS

Como vimos al comenzar a hablar de cursores explícitos es posible utilizar **parámetros**, los cuales permiten que **los valores se transfieran a un cursor cuando éste esté abierto**. El cursor se puede abrir varias veces en un bloque, devolviendo un juego activo distinto cada vez.

Cada parámetro formal de la declaración del cursor debe tener un parámetro real correspondiente en la sentencia OPEN. Los distintos tipos de datos del parámetro son los mismos que los de las variables escalares, pero no se les asignan tamaño. Los nombres de los parámetros son referenciados en la expresión de la consulta del cursor.

Sintaxis :

```

CURSOR nb_cursor [ (nb_parámetro tipo_dato [ { := | DEFAULT } expr ]
....)] [RETURN tipo_return] IS Sentencia SELECT;

```

Ejemplo:

```

DECLARE
    CURSOR EJEMPLO (VAR1 NUMBER) IS
    SELECT * FROM ALUMNOS WHERE MATRICULA > VAR1;
BEGIN
    OPEN EJEMPLO (150000); .....
END;

```

Ejercicio 1:

Solicita un salario mínimo y muestra el nombre, nombre del departamento y salario de los empleados con un salario superior a ese mínimo.

Bucle WHILE:

```

DECLARE
    CURSOR CUR_EMP (SAL_MIN NUMBER) IS
        SELECT E.NOMEMP, D.NOMDEP, E.SALEMP
        FROM EMPLEADO E
        JOIN DEPARTAMENTO D ON E.CODDEP = D.CODDEP
        WHERE E.SALEMP > SAL_MIN;
    V_SALMIN EMPLEADO.SALEMP%TYPE := &SAL_MIN;
    R_CUR_EMP CUR_EMP%ROWTYPE;

BEGIN
    OPEN CUR_EMP(V_SALMIN);
    FETCH CUR_EMP INTO R_CUR_EMP;
    WHILE CUR_EMP%FOUND LOOP
        DBMS_OUTPUT.PUT_LINE(R_CUR_EMP.NOMEMP || '(' || R_CUR_EMP.NOMDEP
|| '): ' || R_CUR_EMP.SALEMP);
        FETCH CUR_EMP INTO R_CUR_EMP;
    END LOOP;
    CLOSE CUR_EMP;
END;
/

```

Bucle FOR:

```

DECLARE
    CURSOR CUR_EMP (SAL_MIN NUMBER) IS
        SELECT E.NOMEMP, D.NOMDEP, E.SALEMP
        FROM EMPLEADO E
        JOIN DEPARTAMENTO D ON E.CODDEP = D.CODDEP
        WHERE E.SALEMP > SAL_MIN;
    V_SALMIN EMPLEADO.SALEMP%TYPE := &SAL_MIN;

BEGIN
    FOR I IN CUR_EMP(V_SALMIN) LOOP
        DBMS_OUTPUT.PUT_LINE(I.NOMEMP || '(' || I.NOMDEP || '): ' ||
I.SALEMP);
    END LOOP;
END;
/

```

3.5 CLÁUSULAS UTILIZADAS EN CURSORES

Hay diferentes cláusulas que podemos utilizar cuando estemos trabajando con cursores. Veamos algunas de ellas.

3.5.1 FOR UPDATE

Esta cláusula se coloca en la SELECT del cursor en la zona declarativa. Permite:

- Bloquear algunas filas antes de la actualización o supresión.
- El bloqueo explícito permite denegar el acceso mientras dura una transacción.

Sintaxis:

```
SELECT .....  
FROM .....  
FOR UPDATE [ OF col_tabla_referenciada][NOWAIT];
```

Donde:

NOWAIT: devuelve error si las filas fueron bloqueadas en otra sesión.

Ejemplo:

Recuperar los pedidos de importes superiores a 1000\$ que se han procesado hoy.

```
DECLARE  
    CURSOR C1 IS  
        SELECT CUSTID, ORDID  
        FROM ORD  
        WHERE ORDERDATE = SYSDATE AND TOTAL>1000.00  
        ORDER BY CUSTID  
        FOR UPDATE NOWAIT;  
...
```

3.5.2 WHERE CURRENT OF

Esta cláusula se utiliza siempre que se haga referencia a la fila actual de un cursor explícito, esto permitirá realizar actualizaciones y supresiones a la fila que se está tratando actualmente.

Se incluirá la cláusula FOR UPDATE en la consulta del cursor para bloquear primero las filas.

Se utilizará la cláusula WHERE CURRENT OF para hacer referencia en la fila actual de un cursor explícito. Esta cláusula normalmente se utiliza en la zona BEGIN . Sintaxis:

```
WHERE CURRENT OF nb_cursor;
```

Ejemplo:

```
DECLARE
    .....
    CURSOR EMP_CURSOR IS
    SELECT .....
    FOR UPDATE; .....
BEGIN
    .....
    FOR EMP_RECORD IN EMP_CURSOR LOOP
        UPDATE .....
        WHERE CURRENT OF EMP_CURSOR;
        .....
    END LOOP;
    COMMIT;
END;
/
```

Imaginad que el cursor es un dedo que señala una fila en una lista. WHERE CURRENT OF es la instrucción que dice: 'Borra/Actualiza exactamente la fila que estoy señalando ahora mismo con el dedo', sin tener que volver a buscarla por su ID